

Machine Learning Foundations: Linear & Logistic Regression

Python | NumPy | TensorFlow | From-Scratch Optimization

Timur Filimonov

Project Overview

This project focuses on the core supervised learning algorithms that form the foundation of modern machine learning systems. The work includes implementing linear regression and logistic regression, understanding cost functions, computing gradients, applying gradient descent, and extending models with polynomial features and regularization.

The project was built to strengthen understanding of both the mathematical logic and practical implementation of regression and classification models.

Imports and Setup

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

Dataset / Input Construction

This project uses simple numerical datasets to demonstrate regression, classification, and feature engineering.

```
x = np.arange(0, 20, 1)
y = np.cos(x / 2)

X = np.c_[x, x**2, x**3, x**4, x**5]
print(X.shape)
```

Polynomial features were created to allow a linear model to capture non-linear patterns.

Linear Regression

Linear regression was implemented to model relationships between input features and continuous target values.

Main concepts covered:

- prediction function
- cost function
- gradient computation
- **gradient descent updates**

Linear Regression Cost Function:

```
def compute_cost(x, y, w, b):  
    m = x.shape[0]  
    cost_sum = 0  
  
    for i in range(m):  
        f_wb = w * x[i] + b  
        cost = (f_wb - y[i]) ** 2  
        cost_sum += cost  
  
    total_cost = (1 / (2 * m)) * cost_sum  
    return total_cost
```

Linear Regression Gradient:

```
def compute_gradient(x, y, w, b):  
    m = x.shape[0]  
    dj_dw = 0  
    dj_db = 0  
  
    for i in range(m):  
        f_wb = w * x[i] + b  
        err = f_wb - y[i]  
  
        dj_dw += err * x[i]  
        dj_db += err  
  
    dj_dw = dj_dw / m  
    dj_db = dj_db / m  
  
    return dj_dw, dj_db
```

Gradient Descent Optimization

Gradient descent was used to iteratively update parameters in order to minimize model cost.

Parameter updates:

$$w = w - \alpha \frac{\partial J}{\partial w}$$

$$b = b - \alpha \frac{\partial J}{\partial b}$$

where α is the learning rate controlling the size of each optimization step.

Example:

```
def gradient_descent(X, y, w, b, alpha, iterations):
    for i in range(iterations):
        dj_dw, dj_db = compute_gradient(X, y, w, b)
        w = w - alpha * dj_dw
        b = b - alpha * dj_db
    return w, b
```

Polynomial Feature Engineering

Polynomial features were added to allow linear regression to model non-linear relationships.

Example implementation:

```
x = np.arange(0,20,1)
y = np.cos(x/2)

X = np.c_[x, x**2, x**3, x**4, x**5, x**6, x**7, x**8, x**9, x**10, x**11, x**12, x**13]
```

This helped demonstrate how linear models can approximate more complex patterns when feature engineering is applied.

Regression Fit Visualization

The figure below shows how polynomial features allow a linear model to approximate a non-linear function.

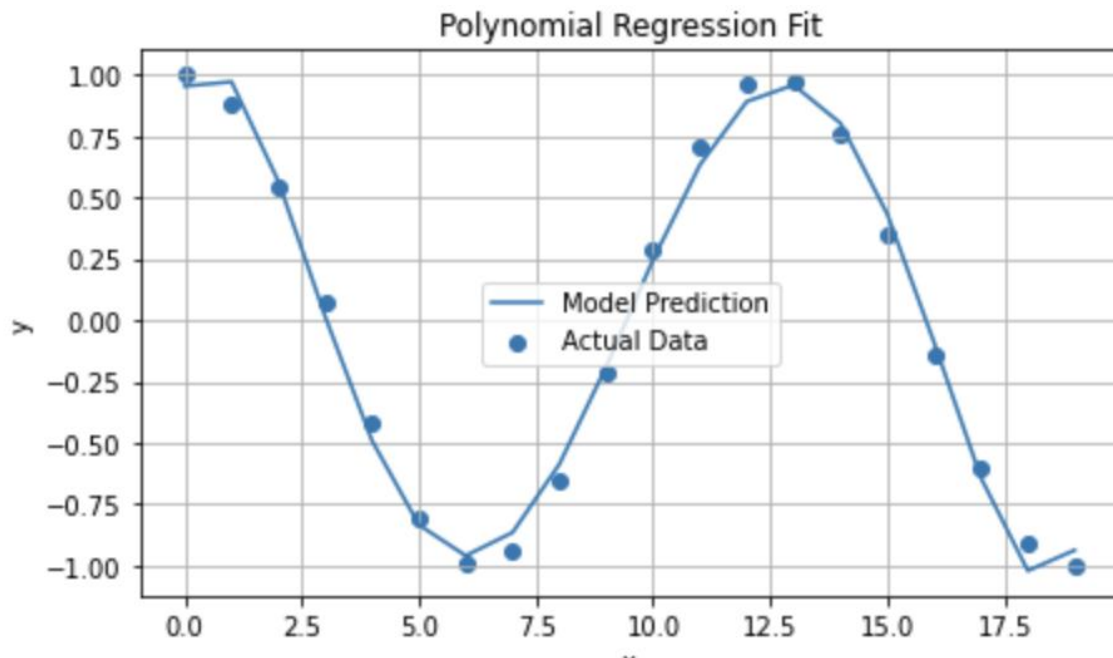


Figure 1. Polynomial regression model approximating a cosine-shaped dataset using engineered features.

Feature Normalization

Feature scaling was applied to improve gradient descent performance and numerical stability.

```
def zscore_normalize_features(X):  
    mu = np.mean(X, axis=0)  
    sigma = np.std(X, axis=0)  
    X_norm = (X - mu) / sigma  
    return X_norm
```

Normalization was especially important for polynomial features with very different numerical scales.

Logistic Regression

Logistic regression was implemented for binary classification tasks.

Main concepts covered:

- sigmoid activation
- logistic cost function
- classification prediction
- decision threshold at 0.5

Sigmoid Function:

```
def sigmoid(z):  
    g = 1 / (1 + np.exp(-z))  
    return g
```

Logistic Cost Function:

```
def compute_cost(X, y, w, b, *argv):  
    m, n = X.shape  
    total_cost = 0  
  
    for i in range(m):  
        z_wb = np.dot(X[i], w) + b  
        f_wb = sigmoid(z_wb)  
        loss = -y[i] * np.log(f_wb) - (1 - y[i]) * np.log(1 - f_wb)  
        total_cost += loss  
  
    total_cost = total_cost / m  
    return total_cost
```

Sigmoid Visualization

```
z = np.arange(-10, 11)
plt.plot(z, sigmoid(z))
plt.xlabel("z")
plt.ylabel("sigmoid(z)")
plt.title("Sigmoid Function")
plt.grid(True)
plt.show()
```

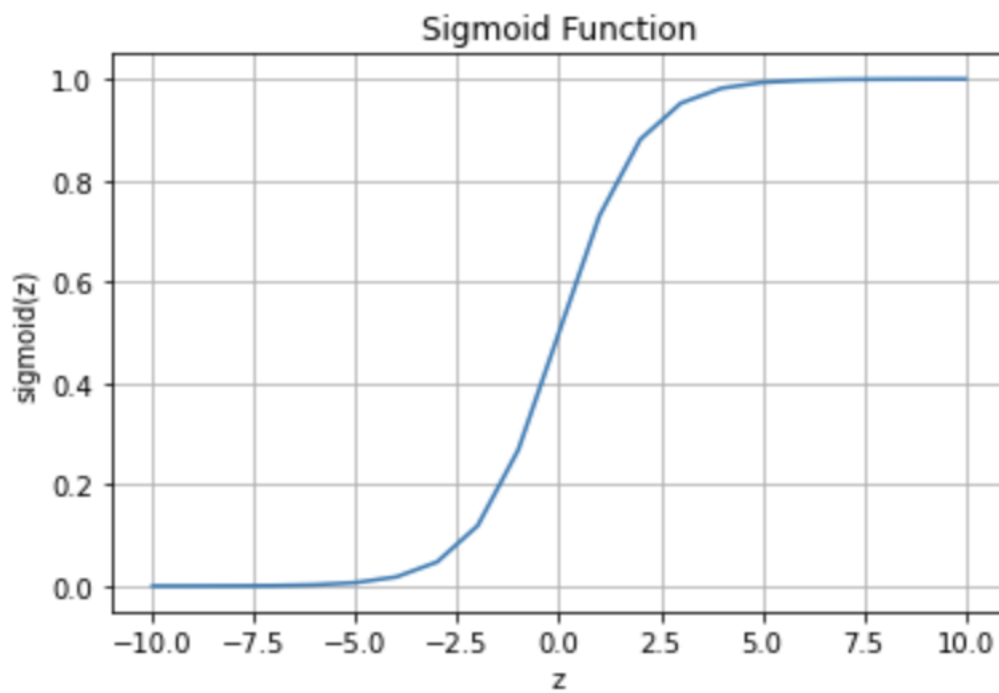


Figure 2. The sigmoid function maps real-valued inputs into probabilities between 0 and 1.

Logistic Regression Gradient:

```
def compute_gradient(X, y, w, b, *argv):
    m, n = X.shape
    dj_dw = np.zeros(w.shape)
    dj_db = 0.

    for i in range(m):
        z_wb = np.dot(X[i], w) + b
        f_wb = sigmoid(z_wb)
        err = f_wb - y[i]

        dj_db += err

        for j in range(n):
            dj_dw[j] += err * X[i, j]

    dj_dw = dj_dw / m
    dj_db = dj_db / m

    return dj_db, dj_dw
```

Prediction Function:

```
def predict(X, w, b):
    m, n = X.shape
    p = np.zeros(m)

    for i in range(m):
        z_wb = 0
        for j in range(n):
            z_wb += X[i, j] * w[j]
        z_wb += b

        f_wb = sigmoid(z_wb)

        if f_wb >= 0.5:
            p[i] = 1
        else:
            p[i] = 0

    return p
```

Regularization

Regularization was introduced to control model complexity and reduce overfitting.

Regularized Cost:

```
def compute_cost_reg(X, y, w, b, lambda_=1):  
    m, n = X.shape  
    cost_without_reg = compute_cost(X, y, w, b)  
    reg_cost = 0.  
  
    for j in range(n):  
        reg_cost += w[j] ** 2  
  
    reg_cost = (lambda_ / (2 * m)) * reg_cost  
    total_cost = cost_without_reg + reg_cost  
  
    return total_cost
```

Regularized Gradient:

```
def compute_gradient_reg(X, y, w, b, lambda_=1):  
    m, n = X.shape  
    dj_db, dj_dw = compute_gradient(X, y, w, b)  
  
    for j in range(n):  
        dj_dw[j] += (lambda_ / m) * w[j]  
  
    return dj_db, dj_dw
```

Key Concepts Demonstrated:

- Linear regression
- Logistic regression
- Cost functions
- Gradient computation
- Gradient descent
- Polynomial features
- Feature normalization
- Binary classification
- Regularization

Key Outcomes

This project strengthened understanding of the mathematical and programming foundations of machine learning, especially the relationship between prediction functions, cost functions, gradients, and optimization. It also demonstrated how regression and classification models can be improved through feature engineering and regularization.